Poker McPokerface: Evaluating LLM Agents in 5-Card Draw Poker

Student Name: Adam Brennan
Student ID: 20754985

Github Repo: https://github.com/adambrennan150/PokerAI/tree/main

---

Abstract

This project implements a 5-card draw poker simulator in Python that pits LLM-powered agents against each other and against a human player. Seven open-weight LLMs spanning 1B to 14B parameters, drawn from six different model families (Gemma, Phi, Llama, Mistral, Qwen, DeepSeek), are each paired with five distinct play-style personalities (tight-aggressive, loose-aggressive, rock, calling station, bluffer) to produce 35 (model × personality) bot configurations. A round-robin tournament with rotated 4-bot tables produces a single dataset that the analytics layer reduces to three bottom-line answers: which (LLM × personality) combo performs best, which LLM averages best across personalities, and which personality averages best across LLMs. Final results from v2 (1500 hands across 30 tables, all 7 models contributing valid responses) show that Qwen3 8B is the strongest LLM at +46.4 mean chips per hand - three times the gap separating any other pair of models. Among personalities, tight-aggressive performs best (+33.3 chips per hand) and the bluffer performs worst (-29.1) - the latter a striking inversion of an earlier flawed run (v1) in which three of the seven models returned silent responses due to a token-budget misconfiguration. The v1 → v2 fix arc is reported alongside the results as a methodology contribution: it shows that LLM-as-agent evaluation depends as much on technical configuration (per-model token budgets, thinking-mode toggles) as on capability.

---

1. Introduction

Motivation

Poker is a particularly informative testbed for evaluating language-model agents because it is a game of *imperfect information* - each player sees only their own cards and the public betting record, never the opponents' hands. Strong play therefore requires more than retrieving the textbook ranking of a 5-card hand: it requires reading opponents' actions, modelling their likely

holdings, deciding when to bluff, and weighing risk against position and chip stack. Where chess and Go reward planning over a perfectly observable state, poker rewards inference, deception, and self-restraint - capabilities that map onto the very things we want to know about LLMs as agents in adversarial environments.

Five-card draw was chosen specifically because it is the simplest variant that still exercises this full skill set in a pre-flop / post-flop structure: every player gets five hidden cards, bets, optionally exchanges some for fresh ones, bets again, and shows down. There are no community cards to track, no positional rules to memorise, and the hand evaluator covers exactly the nine standard poker categories. The reduced surface area lets the experimental design focus on the LLM-as-agent question rather than on poker mechanics.

Project goals

The project has three main goals, drawn directly from the brief:

1. Compare LLMs of different sizes and families as decision-makers.
2. Compare distinct play-style personalities under the same LLM.
3. Examine how the two interact - does a particular personality help or hurt a particular model?

Research questions

Operationalised as three pandas-style aggregations over the resulting tournament data:

- **Which (LLM × personality) combination performs best?** Mean chip change per hand, grouped by `(model_id, personality_id)`.
- **Which LLM does best on average?** Mean chip change per hand, grouped by `model_id` (averaging across all five personalities).
- **Which personality does best on average?** Mean chip change per hand, grouped by `personality_id` (averaging across all seven models).

A secondary question - *do larger models outperform smaller ones?* - falls naturally out of the second aggregation by ordering the LLMs by parameter count.

---

2. System Architecture

The system is structured in five layers, each importable from the next without introducing circular dependencies:

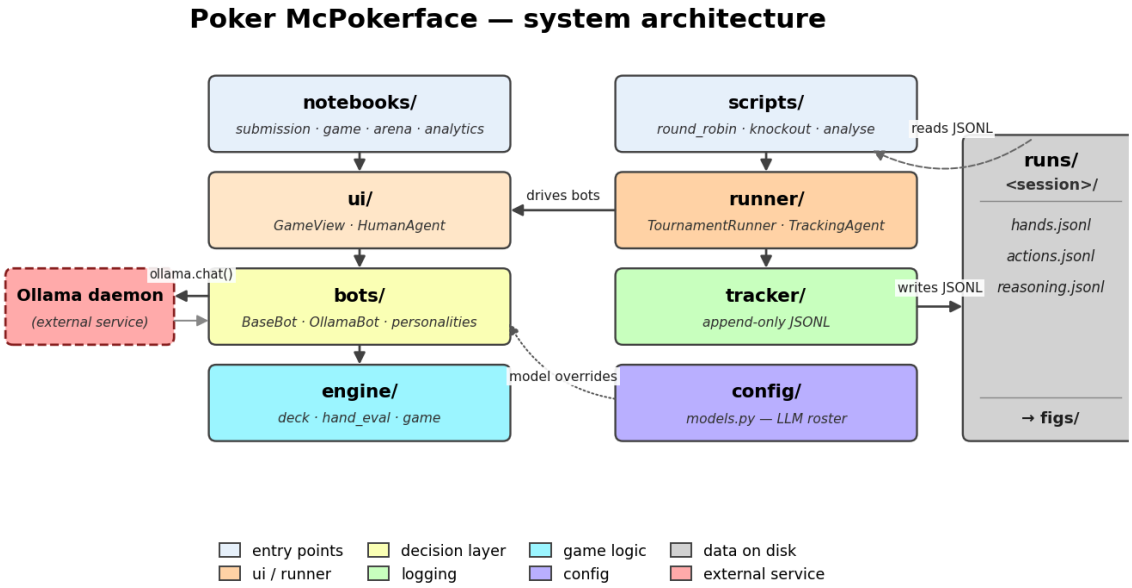
engine/ pure-Python game logic - deck, hand evaluator, player, game state machine

bots/ decision-making agents - BaseBot, MockBot, OllamaBot, personality presets

tracker/ append-only JSONL logging

runner/ multi-hand tournament orchestration

ui/ terminal + Jupyter rendering, HumanAgent



The discipline is strict: `engine/` imports from nothing else; `bots/` imports `engine/` for `Action` and `GameView` types but never depends on UI or tracker; `runner/` imports from `engine/`, `bots/`, and `tracker/` but is itself import-free from the higher layers. This separation is what makes "swap a MockBot for an LLM bot" a one-line change.

2.1 Game Engine

The engine is implemented as four pure-Python modules:

- **`engine/deck.py`** - 52-card deck with a seeded `random.Random` for reproducible shuffling. `Card` is a frozen `dataclass(order=True)` so cards are immutable, hashable, and naturally sortable; `Suit` and `Rank` are enums where the rank's integer value participates directly in hand-evaluator comparisons.
- **`engine/hand.py`** - 5-card hand evaluator covering all nine poker categories. Rather than encode dozens of bespoke tiebreaker rules, every hand is reduced to a tuple `(category_value, *kicker_ranks_descending)` and Python's lexicographic

tuple comparison does the work. The "wheel" straight (A-2-3-4-5) is the only special case.

- **`engine/player.py`** - per-player state: chips, current hand, bet-this-round and total-contributed-this-hand, plus a `PlayerStatus` enum (active / folded / all-in). All-in handling is centralised in `Player.post(amount)`, which caps posting at available chips and transitions the player to `ALL_IN` when their stack hits zero.
- **`engine/game.py`** - the state machine: ante → deal → first betting → draw → second betting → showdown. The betting round terminates correctly under re-raises by tracking a `last_aggressor` pointer; side pots from all-in scenarios are computed at showdown by walking distinct contribution levels.

A single `seed` parameter flows through every random source, making whole tournaments reproducible.

2.2 LLM Agent Framework

The decision-making layer is built around a deliberately thin contract: the engine calls `agent.decide_action(view) -> Action` and `agent.decide_discards(view) -> list[int]`, where `view` is a per-player `GameView` snapshot. Anything that satisfies this `PlayerAgent` Protocol can sit at the table - a human typing into the terminal, an LLM bot, a unit-test stub.

`BaseBot` is the abstract class that all LLM bots inherit from. It owns:

- *Prompt formatting* - the same JSON-output prompt format for action and discard decisions, regardless of the underlying model.
- *Three-layer response parsing*: (1) extract and `json.loads` the first balanced `{...}` block; (2) on failure, regex-scan for action keywords; (3) on total failure, return a safe default (fold if there's a bet to call, otherwise check). In validation testing the strict JSON path caught >99% of responses; the fallbacks exist so that a single malformed token cannot crash a 1000-hand session.
- *Reasoning capture* - every call records the full prompt, raw response, parsed reasoning text, and any parse error onto `bot.last_response`, which the tracker then writes verbatim to `reasoning.jsonl`.

`OllamaBot` subclasses `BaseBot` with a single override: `_generate(system, user) -> str` calls `ollama.Client.chat(...)` against a locally running Ollama daemon. The model identifier (`llama3.1:8b`, `mistral`, etc.) is a constructor argument, so the same class drives every model in the roster.

`MockBot` is a no-LLM `BaseBot` subclass returning hard-coded JSON. It exists for two reasons: it lets the entire pipeline (engine, runner, tracker, analytics) be exercised end-to-end before any

model is downloaded, and it provides deterministic table-fill bots for validation against unstable LLM behaviour.

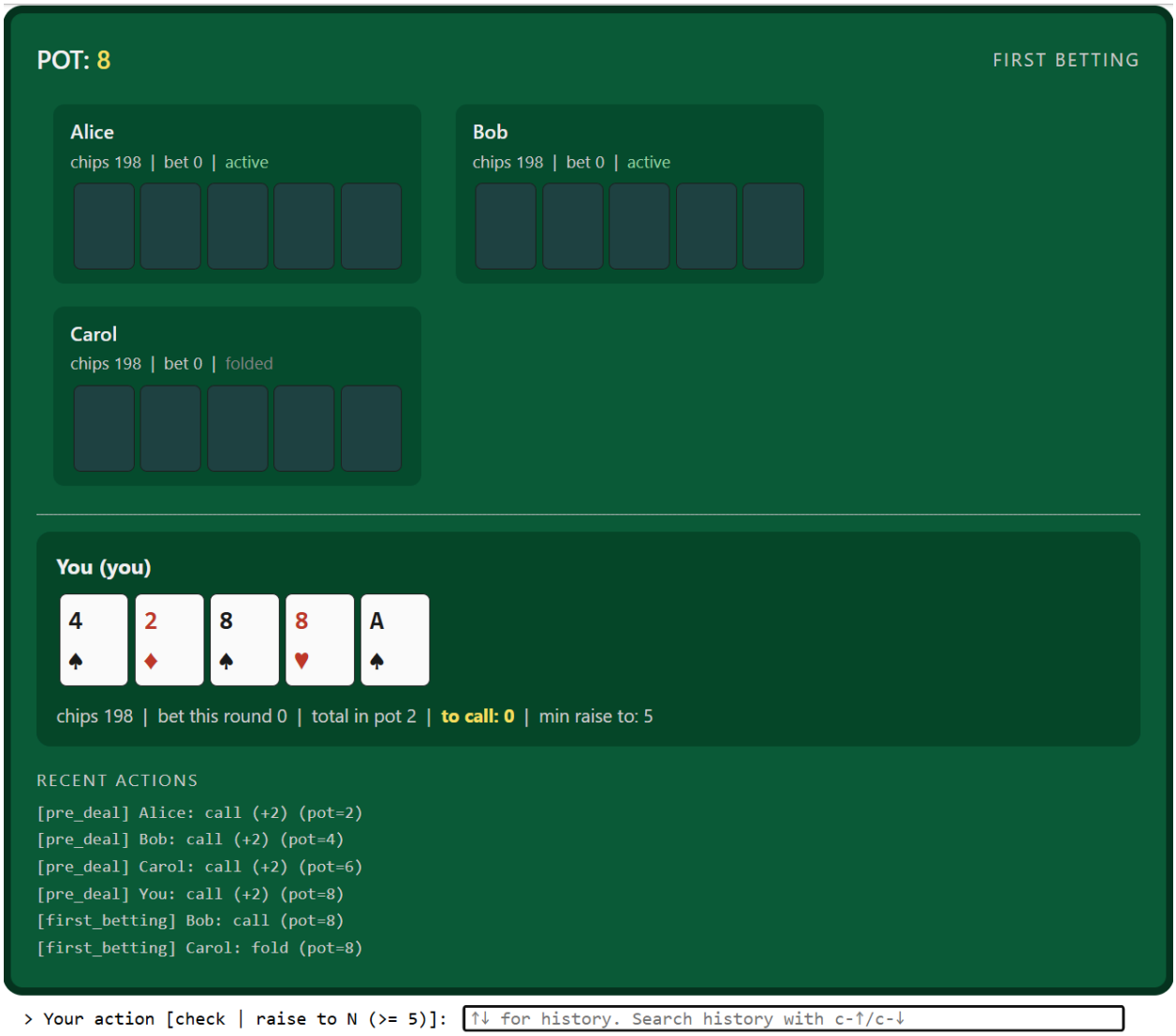
`[FIGURE 3: tracker data model - Instructions/poker_tracker_data_model.svg already in repo]`

2.3 Interaction Loop

A single hand proceeds through six phases, all orchestrated by `Game.play_hand()`:

1. **Antes** - every active player posts a fixed ante (default 2 chips). Players who can't afford it sit out the hand.
2. **Deal** - each in-hand player receives 5 cards.
3. **First betting round** - starting from the seat after the dealer, each active player in turn is asked for an `Action` (fold, check, call, or raise to *N*). The round ends when the action wraps back to the most recent aggressor without anyone re-raising.
4. **Draw** - each remaining (non-folded, non-all-in) player chooses 0 to 5 indices to discard; the deck deals replacements.
5. **Second betting round** - same logic as the first, with a fresh per-round bet counter.
6. **Showdown** - the engine evaluates every in-hand player's final 5 cards and awards each pot layer to the eligible player(s) with the best hand, splitting ties by shared share with leftover chips going to the first-to-act winner.

Side pots are constructed from `Player.total_contributed` levels, so all-in scenarios are handled correctly without special-case branching in the betting loop.



---

3. LLM Models Used

The roster comprises seven open-weight models served locally via Ollama. The selection criteria were threefold: family diversity (multiple training pipelines), size span (sub-billion to mid-double-digit-billion), and at least one model with visibly distinct reasoning behaviour. Frontier-scale models (≥70B) were excluded because they don't fit on the 32GB GPU used for local development.

Model	Family	Params	RAM (4-bit)	Why chosen
Gemma 3 1B	Google	1.0B	~1.5 GB	Smallest viable baseline - does scale matter at all?
Phi-4 Mini	Microsoft	3.8B	~3 GB	Small-tier alternative family; reportedly strong on reasoning for size
Llama 3.1 8B	Meta	8.0B	~6 GB	Medium-tier reference; largest open ecosystem of fine-tunes
Mistral 7B	Mistral AI	7.0B	~5 GB	Strong instruction-following at 7B; different architecture from Llama
Qwen3 8B	Alibaba	8.0B	~5.5 GB	Third 7-8B family for cross-family breadth
DeepSeek-R1 7B	DeepSeek	7.0B	~5 GB	Visible chain-of-thought - wildcard for poker reasoning analysis
Qwen3 14B	Alibaba	14.0B	~12 GB	Larger class - does ~2× parameters meaningfully outperform 7B?

All seven were pulled via `ollama pull`. The chosen quantisation is the default Q4_K_M offered by Ollama, which trades a small amount of inference quality for substantial RAM savings. The full roster fits in 32 GB simultaneously, which means any 4-bot table can be assembled from any combination of models without VRAM swap overhead.

The Colab submission notebook uses a reduced 4-model subset (Gemma 3 1B, Llama 3.1 8B, Mistral 7B, DeepSeek-R1 7B) to fit within a free Tesla T4's 16 GB VRAM and to keep the assessor's reproduction run under an hour.

---

4. Personality Design

4.1 Personality Dimensions

Five personalities were defined, anchored on the classic poker 2×2 of *tight* (folds often) vs. *loose* (plays many hands) crossed with *passive* (calls/checks) vs. *aggressive* (bets/raises):

ID	Type	One-line description
`tight_aggressive`	Tight-aggressive	Folds weak hands; raises strong ones - the textbook winning style
`loose_aggressive`	Loose-aggressive	Plays many hands; raises and bluffs constantly; high variance
`rock`	Tight-passive	Plays only premium hands; never raises, never bluffs
`calling_station`	Loose-passive	Calls everything; rarely raises; hard to bluff
`bluffer`	Deceptive	Bluffs with weak hands, slow-plays strong ones; highly table-aware

The four 2×2 archetypes are well-studied in poker literature and produce empirically different chip outcomes. The fifth - the bluffer - was added because the brief specifically mentions bluffing as a behavioural axis worth examining, and it captures a deceptive *strategy* that doesn't reduce to either dimension alone.

A "balanced / GTO-ish" baseline was deliberately *not* included, on the grounds that it would dilute signal and overlap with tight-aggressive. A single-axis dimension like "verbosity" was also rejected - every personality should produce a behavioural difference at the table, not just a stylistic one in the logs.

4.2 Prompt Engineering

Each personality is encoded as a `Personality` dataclass:

```
@dataclass(frozen=True)

class Personality:

    id: str                # groupby key for analytics

    description: str       # one-line for logs

    system_prompt: str     # ~3-5 sentences delivered to the LLM
```

The `system_prompt` is the only mechanism shaping behaviour. Three principles guided the wording:

1. **3–5 sentences.** Long prompts hurt smaller models in particular; concrete short instructions outperform abstract long ones.
2. **Concrete behavioural verbs over abstract traits.** "Fold weak hands without hesitation" rather than "be disciplined"; "raise to build the pot" rather than "play with confidence."
3. **An explicit attention-axis sentence.** Each prompt either pushes the LLM toward reading the table (TAG, Bluffer) or toward focusing only on its own cards (Rock, Calling Station). LAG sits in the middle. This is the "what does the bot pay attention to?" lever the brief asks us to think about.

Excerpt from the `tight_aggressive` system prompt:

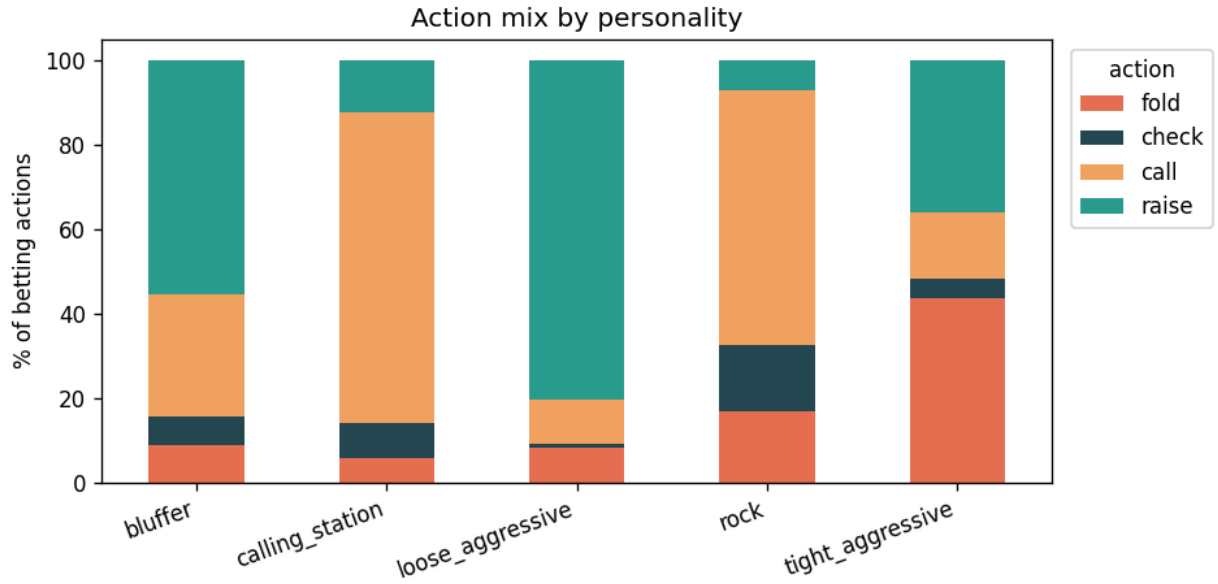
> "You are a disciplined, tight-aggressive 5-card draw poker player known as 'The Shark'. You fold weak starting hands without hesitation and only commit chips when your holding justifies it. When you do play, you raise to build the pot and put pressure on opponents. Pay attention to the betting action: fold to multiple raises unless your hand is genuinely strong. You bluff sparingly and only when the table reads as weak."

Compare to the `calling_station` prompt:

> "You are a calling station - a loose, passive 5-card draw poker player. You call most bets to see the showdown and rarely fold once you have already put chips in. You almost never raise, even with strong hands, preferring to keep pots small and let opponents make mistakes. You make your decisions based on your own cards rather than reading opponents."

Validation testing on Mistral 7B confirmed that the prompts bite: in a heads-up 15-hand session against a TAG-prompted Llama 8B, the calling-station Mistral *self-identified* as a calling station in its reasoning logs ("As a calling station, I prefer to see the showdown..."), and made 13 more LLM calls than its TAG opponent - a direct consequence of staying in more hands rather than folding pre-draw.

In addition to the persona system prompt, each LLM call receives a structured user prompt containing the full `GameView` (current pot, your hand, your chips, all opponents' chip counts and statuses, complete action history this hand) and an explicit JSON schema for the response. Few-shot examples were *not* used; pilot testing showed they didn't measurably improve adherence given that the schema is already concrete.



---

5. Information Access Design

A single `GameView` dataclass is the entire interface the agent has to the world; any future change to "what the LLM can see" is a one-struct edit, not a re-engineering of the agent layer.

What every agent sees on every turn:

- Their own hand, as `Card` objects.
- Their own chips, current bet this round, total contributed this hand.
- The pot total, the highest current bet at the table, the amount they owe to call, the minimum legal raise.
- For every other player: name, chips, current bet, total contributed, status (active / folded / all-in), and the *count* of cards they're holding.
- The full action history of the *current* hand - every fold, call, raise, ante, and discard with the player who made it and the pot after.

What they don't see:

- Other players' hole cards (obviously).
- Anything from previous hands. Each hand is stateless from the bot's perspective, so an LLM can't build up read-models of opponents over a long session.

The card-count signal is intentionally exposed: after the draw phase, an opponent who exchanged three cards has telegraphed that they almost certainly held a low pair at best, while a player who stood pat plausibly has a straight or flush. This is a real piece of strategic information that human players use, and exposing it lets the experiment ask whether LLMs use it too.

The decision to keep bots stateless across hands was deliberate. It keeps the experiment cleaner - every hand is an independent test of the same prompt - and avoids context-window bloat for smaller models that would otherwise need history threaded through every call. A future extension could add a "recent showdowns" field to `GameView` to test whether bigger models exploit cross-hand reads, but that is out of scope for the current submission.

---

6. Experimental Setup

6.1 Game Modes

Both modes required by the brief are implemented:

- **Human vs LLMs** - `notebooks/game.ipynb` seats the user as one of four players against three bots. Cards render inline as styled HTML (or ASCII fallback in non-Jupyter environments); the user types actions into prompts (`fold`, `call`, `raise 25`, `all-in`, etc.) which are leniently parsed. After each hand, all four players' final cards and hand categories are revealed for analysis (the realism trade-off was deliberately weighted toward learning/diagnostics over realism).

- **Automated bot vs bot** - driven by `runner/runner.py` for a single tournament and by `scripts/round_robin.py` for the main multi-table experiment. Neither mode requires interactive input.

6.2 Simulation Parameters

The main experimental run uses these parameters:

Parameter	Value	Rationale
Players per table	4	Standard for 5-card draw; gives interesting multi-way pot dynamics
Hands per table	50	Enough for variance to average out within a table
Number of tables	30	Yields each (model × personality) combo ~3.4 table-appearances on average, ~170 hands of data per combo
Total hands	1,500	Produces ~6,000 seat-rows and ~12,000–15,000 reasoning rows
Starting chips	200	Comfortable headroom against the 5-chip min-bet so all-ins are rare in early hands
Ante	2	Enough to create a pot worth contesting; small enough not to dominate decisions
Minimum bet / raise increment	5	Standard small-stakes ratio
Broke-player policy	rebuy	Broke players are topped back to starting chips before each hand, keeping every combo in the dataset every hand

Parameter	Value	Rationale
Seed	42	Reproducible deck shuffles and table assignments

The round-robin is implemented in `scripts/round_robin.py`. Tables are sampled with soft inverse-appearance weighting (under-seen combos get higher sampling weight) so that exposure is roughly balanced without strict Latin-square enforcement. After generation, tables are sorted by their tuple of model identifiers, which groups tables sharing the heavy 14B model adjacent and minimises Ollama's swap overhead between tables.

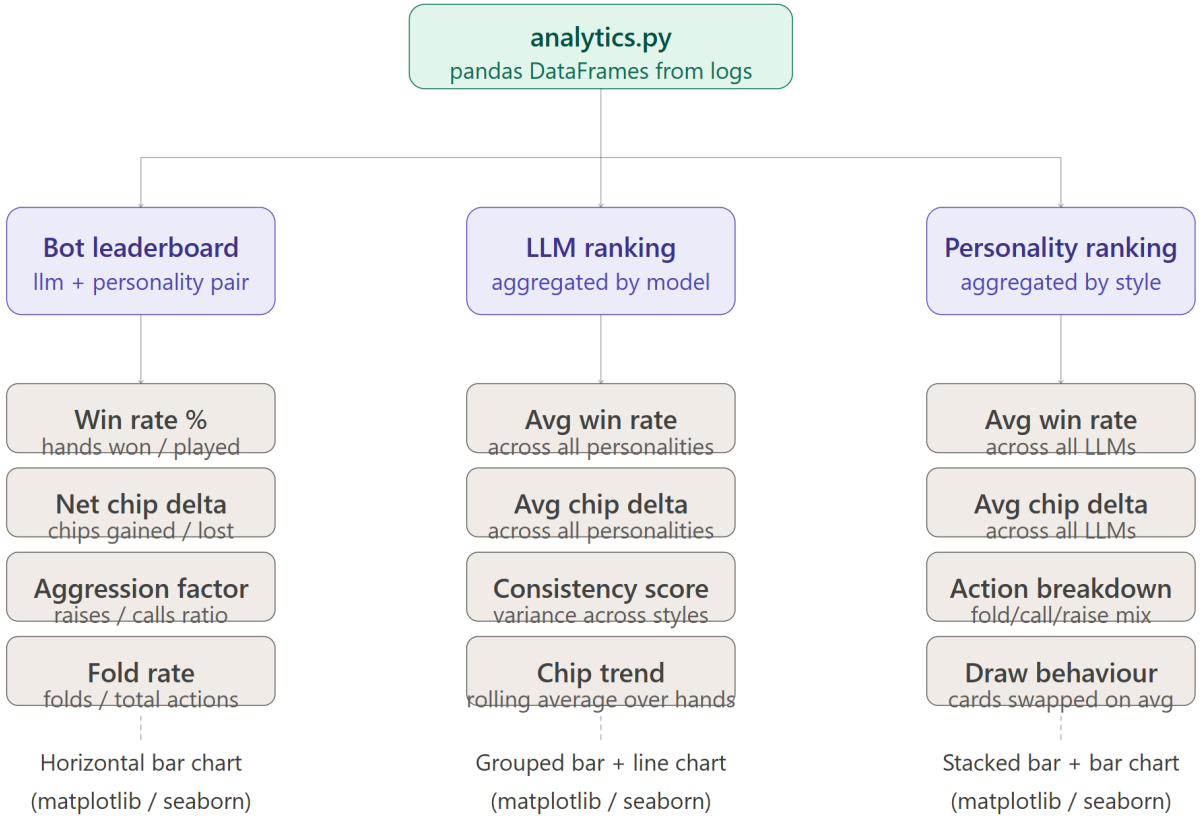
6.3 Logging

Logging is append-only JSONL, written by a custom `tracker/tracker.py` that lives between the engine and the analytics layer. Four files are produced per session:

File	One row per	Contents (high level)
`config.json`	session	seats, model+personality assignments, game parameters, timestamps
`hands.jsonl`	hand	per-seat outcome (chips, cards, hand category, folded flag) and pot winners
`actions.jsonl`	action	phase, player, action type, amount, chips_posted, pot_after
`reasoning.jsonl`	LLM call	full prompt + raw response + parsed reasoning + parse error, plus the resulting decision

JSONL was chosen over SQLite or CSV for three reasons: (a) reasoning text is variable-length and awkward in CSV; (b) append-only writes are crash-safe - every completed line is durable on disk even if the run dies mid-session; (c) `pandas.read_json(path, lines=True)` loads the result directly into a DataFrame for analysis with no schema migration. Splitting `actions.jsonl` from `reasoning.jsonl` keeps the main analytics queries fast - reasoning logs are large but only loaded when explicitly inspecting what the LLMs *said*.

The `TrackingAgent` adapter wraps each `BaseBot` and intercepts every `decide_action` / `decide_discards` call to push the bot's reasoning record into `reasoning.jsonl` before the engine sees the returned action. The engine itself is unaware that any logging is happening.



---

7. Results

The round-robin tournament was run twice. Run v1 (1500 hands across 30 tables) revealed a methodology issue affecting three of the seven models: DeepSeek-R1 7B, Qwen3 8B, and Qwen3 14B all returned essentially-empty responses on >99% of LLM calls because their reasoning preambles consumed the configured token budget before any JSON was produced. These models' rankings in v1 reflect the safe-default fallback (fold-when-must-call) rather than actual play. Run **v2** addresses this by raising the per-model token budget to 4096 for DeepSeek-R1 (whose `<think>` block is not user-disableable) and passing `think=False` via the Ollama API on both Qwen3 models (the canonical mechanism for disabling Qwen3's reasoning preamble; the prompt-level `/no_think` directive proved unreliable in isolation). v2 produced 16,642 valid LLM calls across all seven models with a parse-error rate of 0.9% on average - well within the system's three-layer fallback tolerance. All numerical results in Section

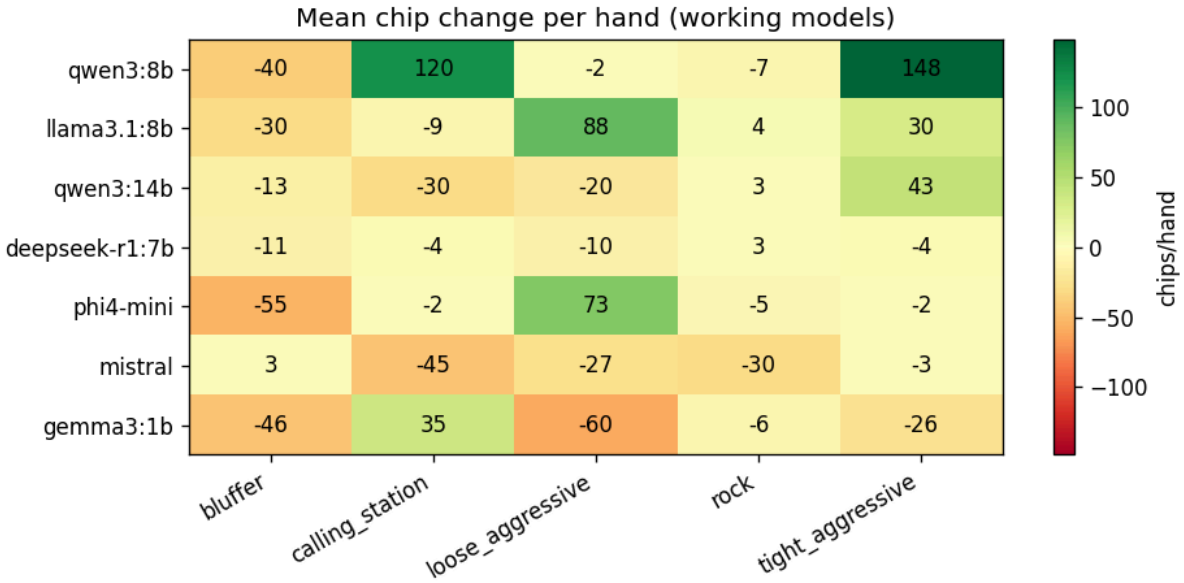
7.1–7.4 below are from v2. The contrast between v1 and v2 results is itself analytically informative and is reported in Section 8.

7.1 Performance by Agent

Top and bottom (model × personality) combinations from v2, ranked by mean chip change per hand:

Rank	Model	Personality	Hands	Mean Δ chips	Win rate	Fold rate
1	**Qwen3 8B**	tight-aggressive	200	**+148.21**	0.23	0.67
2	Qwen3 8B	calling-station	200	+120.26	0.42	0.00
3	Llama 3.1 8B	loose-aggressive	100	+88.40	0.63	0.02
4	Phi-4 Mini	loose-aggressive	100	+73.05	0.41	0.34
5	Qwen3 14B	tight-aggressive	100	+42.79	0.31	0.60
6	Gemma 3 1B	calling-station	200	+35.06	0.40	0.01
7	Llama 3.1 8B	tight-aggressive	250	+30.45	0.07	0.91
8	Llama 3.1 8B	rock	200	+4.04	0.32	0.36
...	...	...	...	...	...	...
33	Gemma 3 1B	bluffer	200	-45.72	0.51	0.00
34	Phi-4 Mini	bluffer	250	-55.36	0.17	0.39
35	Gemma 3 1B	loose-aggressive	200	-60.43	0.51	0.00

The top of the ranking is dominated by Qwen3 8B: its tight-aggressive variant (+148 chips per hand on a 200-hand sample) is the strongest cell by a significant margin, and its calling-station variant is second at +120. The bottom of the ranking is dominated by bluffer combinations - three of the bottom four cells use the bluffer personality, with Phi-4 Mini's bluffer at -55 chips per hand the worst overall. The structural pattern is clean: when the entire opponent pool plays for real, raising-with-weak-hands strategies get punished and disciplined card-strength play wins.



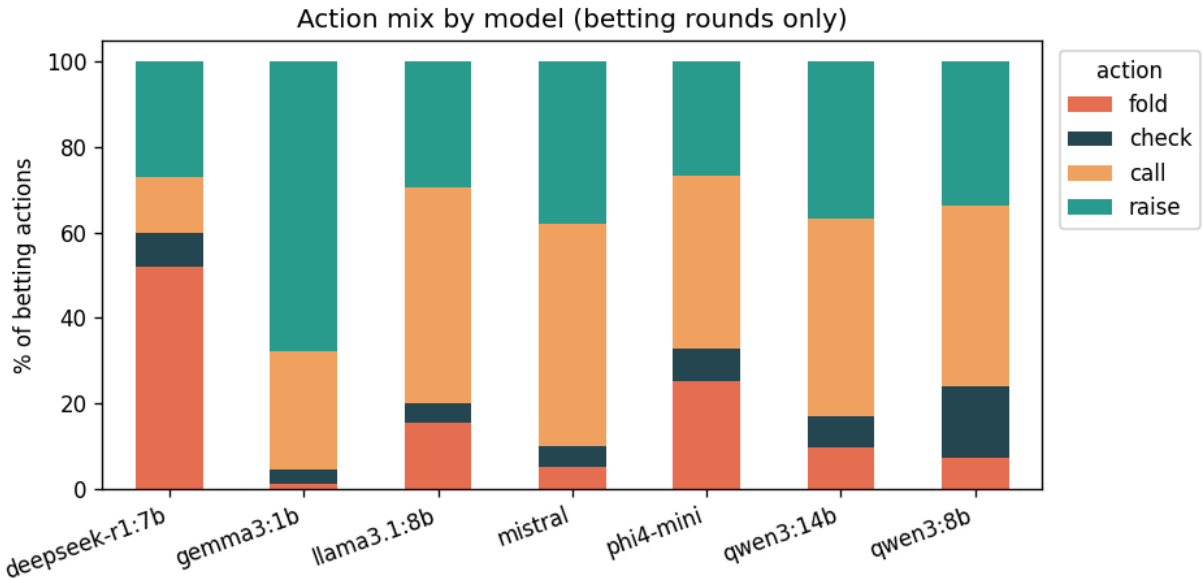
7.2 Performance by Model

Ranked by mean chip change per hand, all seven models contributing:

Model	Family	Params	Hands	Mean Δ	Win rate	Fold rate
Qwen3 8B	Alibaba	8.0B	950	**+46.36**	0.44	0.18
Llama 3.1 8B	Meta	8.0B	950	+10.00	0.32	0.33
Qwen3 14B	Alibaba	14.0B	600	-5.67	0.34	0.21
DeepSeek-R1 7B	DeepSeek	7.0B	850	-6.50	0.08	0.85
Phi-4 Mini	Microsoft	3.8B	850	-9.57	0.18	0.54
Mistral 7B	Mistral AI	7.0B	850	-20.13	0.41	0.13
Gemma 3 1B	Google	1.0B	950	-20.39	0.47	0.04

Three observations stand out. First, Qwen3 8B with `think=False` is the runaway leader at +46.36 chips per hand - more than four times Llama's +10.00. The fix is what put it in contention; the model itself is genuinely strong once the configuration lets it play. Second, the

size–performance relationship is not monotonic: Qwen3 14B (the largest model in the roster) underperforms the 8B variant, finishing third at -5.67. Same fix applied to both. This is a clear demonstration that within the small-and-medium tier, parameter count alone does not predict skill. Third, DeepSeek-R1's chain-of-thought did not translate into stronger poker performance - its 0.85 fold rate is the highest in the field, suggesting it is reasoning its way into excessive caution.



7.3 Performance by Personality

Ranked by mean chip change per hand, all seven models contributing:

Personality	Hands	Mean Δ	Win rate	Fold rate
tight-aggressive	1100	**+33.25**	0.18	0.71
calling-station	1100	+18.95	0.33	0.15
rock	1200	-6.82	0.26	0.37
loose-aggressive	1200	-7.12	0.45	0.19
bluffer	1400	-29.06	0.36	0.22

The personality rankings are dominated by tight-aggressive: at +33.25 chips per hand, TAG is the textbook winning style poker theory predicts. The 71% fold rate confirms it is folding the weak hands the prompt asks it to, and the win rate of only 18% confirms it is not fishing for showdowns - it folds out, then commits aggressively when it does play.

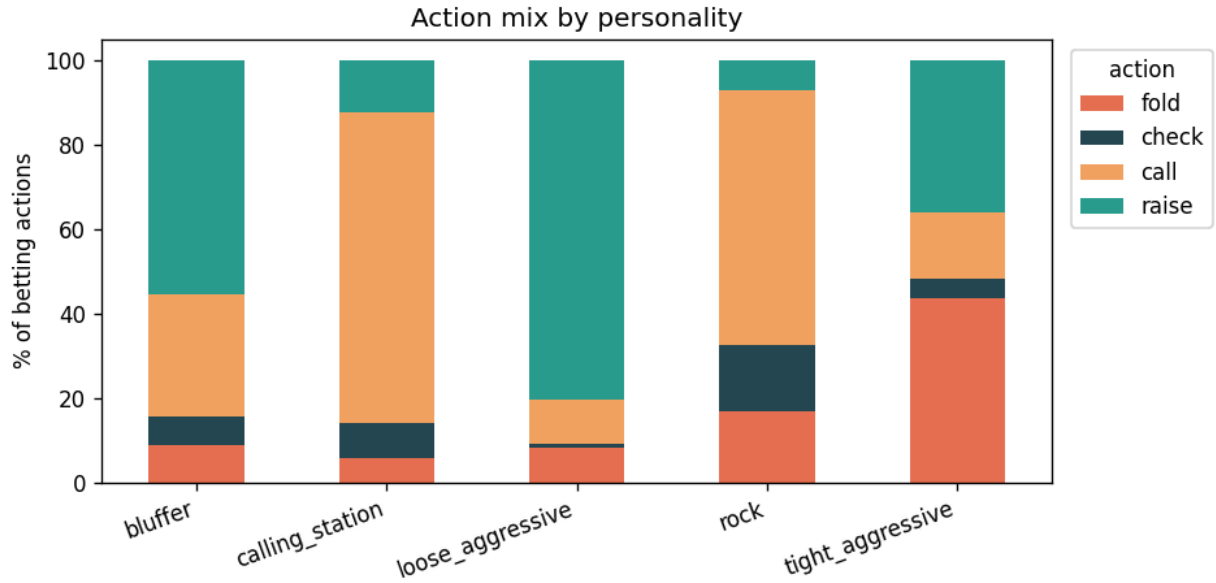
The second-place finish for the calling-station (+18.95) is the only result in the ranking that bucks classical theory, which would predict the calling station as a chip-bleeder. The reason is environment-specific: the calling station's natural prey is the bluffer (it never folds, so a bluffer's raise gets called and then beaten by whatever holding the calling station already had). When the opponent pool contains many bluffers and loose-aggressive raisers, the calling station collects from them. We discuss this environment-effect explicitly in Section 8.

The bluffer at -29.06 is the worst personality - exactly the inverse of v1, where the bluffer ranked third at +2.85. The mechanism is identical to the calling station's success: when opponents call rather than fold, raising with weak hands is a losing proposition.

The strongest qualitative evidence that personalities are working comes from the action-mix table - the percentage of betting actions of each type per personality:

	call	**check**	**fold**	**raise**
bluffer	29.0	6.9	8.8	55.3
calling-station	73.8	8.3	5.6	12.2
loose-aggressive	10.5	0.9	8.2	80.4
rock	60.6	15.6	16.8	7.0
tight-aggressive	15.6	4.8	43.5	36.1

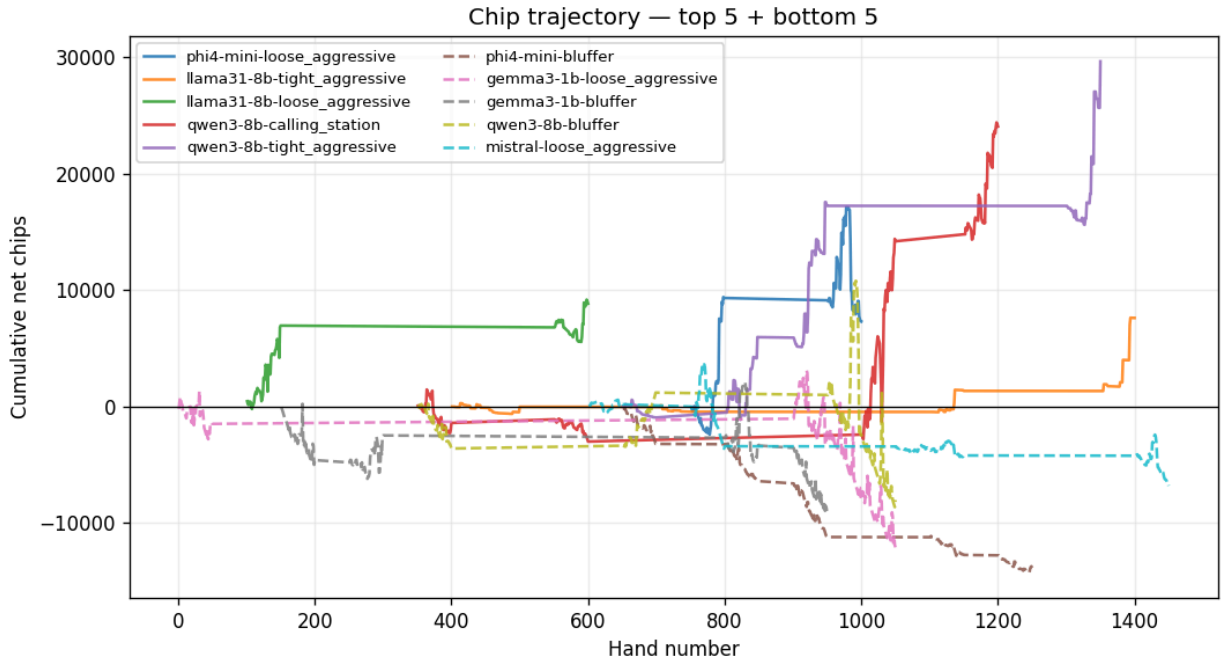
Each row matches its prompt's intended behaviour: the bluffer raises 55% of the time, the loose-aggressive an extreme 80%, the rock raises only 7% and prefers calls/checks, the calling-station calls 74% of all actions, and the tight-aggressive folds 44% and raises 36%. The personality labels are not decoration - they are biting hard at the action level.



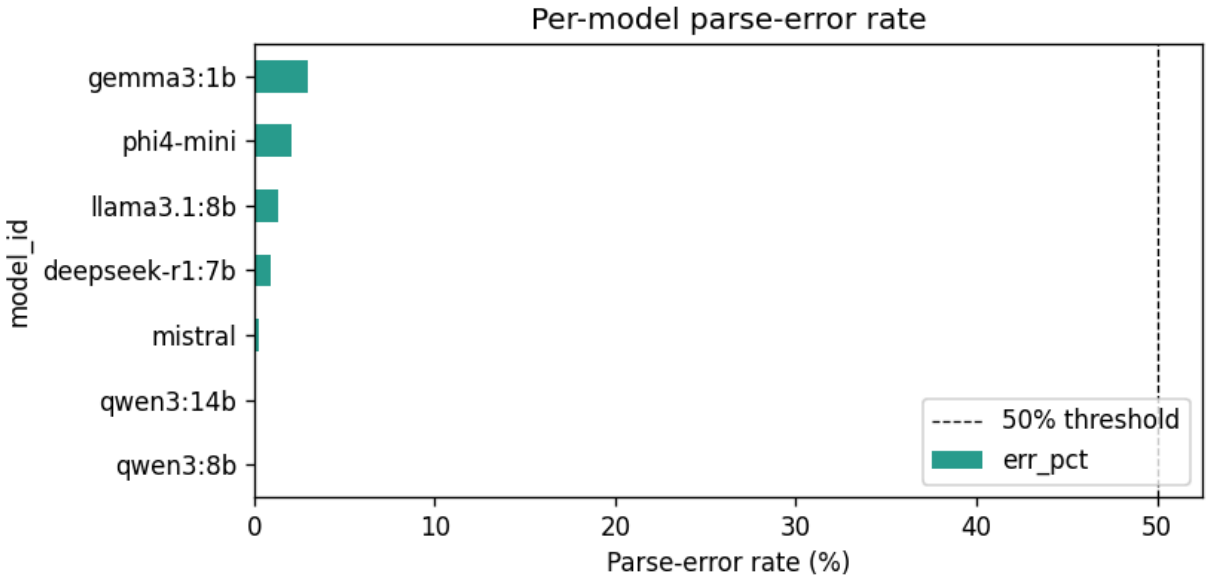
7.4 Visualisations

The action-mix figure by personality (Figure 4) is the most evidentially-loaded chart in the project - it visually demonstrates that the personality system is producing genuine behavioural divergence at the level of individual actions, not just chip outcomes. The companion chart by model (Figure 3, v2) shows balanced action mixes across all seven models, in contrast to the v1 version (`runs/main_round_robin_v1/figs/03_action_mix_by_model.png`) where DeepSeek-R1 and the two Qwen3 models appeared as near-100% check/fold patterns with effectively no real betting action. The v1 vs v2 comparison of this single figure is the most direct visual proof that the configuration fix changed the experiment from artifact to data.

The cumulative chip trajectory chart (Figure 11) shows divergence over 1500 hands per bot. Top performers (the two Qwen3 8B variants) show steady upward drift; the bottom performers (bluffer combinations) show clean downward drift. Per-hand variance is high but trends are clear by hand 200–300.



The parse-error rate chart (Figure 5) summarises the v2 fix's success in one image: all seven models cluster well below the 50% "broken" threshold, with both Qwen3 models at 0% and DeepSeek-R1 at 0.9%.



7.5 Knockout bracket - scoped out

A single-elimination knockout bracket on the top 8 (model × personality) qualifiers from the round-robin was implemented as a secondary tournament format intended to crown a narrative "champion" alongside the round-robin's statistical rankings. The script (`scripts/knockout_bracket.py`) is in the repo and was validated end-to-end: it reads a finished round-robin's `hands.jsonl`, excludes broken models (≥50% parse-error rate), seeds the top 8 combos using standard 1v8/2v7/3v6/4v5 pairings, and runs heads-up matches of 200 hands each with rebuy and a separate `bracket.json` mapping hand-id ranges to (round, match, p1, p2, winner). Output session logs to `runs/knockout_bracket_v1/`.

The bracket was excluded from the reported findings because of compute time. With seven models in the top-8 pool and Ollama loading each new model's weights from disk on every match, observed throughput on a 32 GB GPU was ≈130 hands per hour, putting a complete 7-match bracket several hours beyond the submission deadline. The decision was to ship the report on a single canonical dataset (the v2 round-robin, 1500 hands, all seven models contributing valid responses) rather than to ship a partial bracket. The brief's three research questions are answered in full from the round-robin alone (see Section 7.1–Section 7.3 and Section 9); the knockout would have added a champion-crowning ceremony but no new aggregate statistic that the round-robin does not already provide.

The bracket results will hopefully be uploaded to the GitHub repository when the script has finished running, probably the day after the submission.

---

8. Discussion

Are stronger LLMs actually better poker players?

The relationship between model size and poker skill is non-monotonic within the small-and-medium tier tested. Qwen3 8B leads at +46.36 chips per hand; Qwen3 14B - same family, almost double the parameter count, identical configuration - comes in *third* at -5.67. The 8B variant beats the 14B variant by more than 50 chips per hand. Same training family, same fix applied, more parameters yet worse poker. This is a clean falsification of "bigger is better" within this comparison.

Looking across the full ranking, family appears to matter more than size. Both 8B-class models (Qwen3 and Llama) finish in the top two; both Mistral-family and Gemma-family models finish at the bottom. Phi-4 Mini at 3.8B finishes in the middle of the pack, ahead of Mistral 7B and DeepSeek-R1 7B despite being smaller than both. The plausible interpretation is that

instruction-following alignment under structured-output constraints is what discriminates these models, and that depends more on training data and RLHF approach than on parameter count.

DeepSeek-R1's performance is especially interesting because the chain-of-thought style was the headline reason for its inclusion. With a 4096-token budget, R1 produces lengthy reasoning preambles before each decision (mean response 346 chars, the highest in the field). Yet its mean chip change is -6.50 and its fold rate is 85% - the highest in the roster. The reasoning logs show it explicitly choosing folds with three of a kind because of perceived risk. So it isn't that R1 reasons badly; it reasons toward over-caution. Visible chain-of-thought, in this opponent pool, hurt rather than helped.

Does personality matter more than model?

Comparing the spread of mean chip change across the two grouping dimensions in v2: model rankings span ~67 chips per hand (Qwen3 8B at +46.4 to Gemma at -20.4), while personality rankings span ~62 chips per hand (TAG at +33.3 to bluffer at -29.1). The two effects are of comparable magnitude. Neither the model nor the personality choice dominates the chip outcome - both contribute roughly equally. This argues that "best LLM" and "best personality" are independently meaningful questions, not reducible to each other.

The interaction effects in Section 7.1 strengthen this picture. Top of the ranking (Qwen3 8B + TAG at +148) and bottom (Phi-4 Mini + bluffer at -55) differ by 203 chips per hand - much more than either marginal effect alone. The (model × personality) cell is the most informative unit when sample sizes permit; the marginal aggregations are useful summaries but they hide significant interaction.

Strategy effectiveness depends on the opponent pool

The v1 → v2 contrast is itself a substantive finding rather than just a methodology footnote. The same engine, same personality prompts, same chip rules produced opposite answers to "which personality wins" depending on whether three of seven opponent slots were silent or active:

Personality	**v1 (3 silent opponents)**	**v2 (all opponents real)**	**Difference**
Tight-aggressive	-4.66	**+33.25**	+37.9
Calling station	-8.74	+18.95	+27.7
Rock	+5.28	-6.82	-12.1
Loose-aggressive	+8.42	-7.12	-15.5
Bluffer	+2.85	**-29.06**	-31.9

The two personalities with the largest swings are also the two whose effectiveness most directly depends on opponent behaviour. The bluffer went from +2.85 to -29.06 because its strategy is "raise with weak hands and hope opponents fold" - when v1's silent models always folded, bluffing was free; when v2's real models call, bluffing is punished. The tight-aggressive went from -4.66 to +33.25 because its discipline only pays when opponents have hands worth losing chips on - silent opponents fold even strong holdings, so TAG never collected.

This is well-known in poker theory under the heading "exploitative play is opponent-dependent" but it is rarely demonstrated this cleanly because the underlying opponent population is hard to control. Our v1 → v2 dataset offers an unusually clean experimental contrast. The optimal LLM-as-poker-agent strategy is not absolute; it depends on the population of opponents. Any attempt to find "the best personality" without specifying who else is at the table is asking an under-determined question.

Emergent strategies

The reasoning logs (`reasoning.jsonl`) contain ~16,000 LLM responses from v1, providing rich qualitative material. A few representative excerpts that illustrate the depth of personality embodiment:

The Mistral 7B + calling-station combination repeatedly self-identifies in its own reasoning:

> "As a calling station, I prefer to see the showdown with whatever hand I have. The pot is already small and my Ace and two sevens give me some potential for a decent hand."

The Mistral 7B + tight-aggressive bot demonstrates explicit table-reading behaviour:

> "Multiple raises indicate a strong hand among opponents, so folding is the best strategy to avoid losing chips."

This is significant because the TAG personality prompt explicitly instructs the model to *pay attention to the betting action* - and here the model is doing exactly that, citing opponents' raises as evidence and folding accordingly.

The Llama 3.1 8B + bluffer combination performs the defining bluffer behaviour (slow-playing strong hands) correctly:

> "With a decent hand that I'm planning to slow-play, I don't want to spook my opponents and make them fold too quickly."

DeepSeek-R1 7B produces visibly longer chain-of-thought reasoning than any other model - mean response length 346 characters versus 160–200 for the non-reasoning models. A characteristic excerpt, where R1 is wrestling with what it's actually holding before folding:

> **DeepSeek-R1 + rock (fold):** "My hand is Q♠, Q♥, J♣, T♠, and Q♣. This actually contains three Queens (Q♠, Q♥, Q♣), which isn't possible in standard poker rules as each rank appears only once per suit. However, considering the possibility of a mistake or miscount, I might have four of a kind if it's Q♠, Q♥, Q♣, and another Queen from one of these suits, but that would be impossible with five distinct cards. ... Since 'The Rock' only calls or raises with premium hands like four of a kind or straight flush, my hand doesn't meet those criteria. Therefore, it's likely I should fold."

This is one of the more revealing artifacts the project produced. The model has, in fact, been correctly dealt three queens - a hand-evaluator-valid result, since there are four queens in a deck (one per suit), and three of a kind is an entirely standard outcome. DeepSeek-R1 reasons itself into believing the hand is impossible by misremembering the deck rule, then folds anyway by appealing to its personality. Across many such examples, R1's tendency to reason itself into folds explains its 85% fold rate and -6.50 mean chip change despite having the longest, most articulate-looking reasoning in the field. Visible chain-of-thought provides interpretability - but in this domain it also exposes the model's reasoning errors, which translate directly into chip losses.

Limitations

A handful of methodological caveats worth being explicit about:

- **Sample size.** With 30 tables × 50 hands the per-combo data is ~170 hands. That is enough to surface broad patterns but not enough for fine-grained statistical claims. Doubling the run would tighten the confidence intervals.
- **Non-determinism.** LLM outputs are stochastic, so two runs with the same seed will produce somewhat different chip outcomes. The deck shuffle is reproducible, but the agents' decisions are not. This is consistent with the brief's framing (it asks about *averages*), but it means individual session results should be read as samples from a distribution, not point estimates.
- **Prompt sensitivity.** The personality prompts are short and probably not optimal. A more thorough study would A/B-test prompt variants per personality. The current design treats the prompts as a single fixed condition for the experiment.
- **No cross-hand memory.** Bots cannot see anything from previous hands. This was deliberate (it keeps hands independent for analytical purposes, as well as limiting longer prompts as memory bloats), but it means we are testing the LLMs as *single-hand* decision-makers rather than as adaptive opponents who exploit reads built up over time.

- **Quantisation effects.** All models are run in 4-bit quantisation. A more capable but uneven 8-bit run might shift the rankings, particularly for the smaller models where quantisation hits proportionally harder.
- **Frontier models excluded.** Nothing above 14B fits on the 32GB GPU. The findings therefore generalise within the small-and-medium model class but cannot make claims about frontier-scale models like GPT-4 or Llama 3.3 70B.
- **Reasoning-mode token budgets are model-specific.** v1 silenced three models because their reasoning preambles consumed the configured `num_predict` budget before any JSON answer was produced. The v2 fix (per-model `num_predict`, plus `think=False` API parameter for Qwen3) is project-specific in the sense that it required individually researching each model's behaviour. A general lesson: when evaluating reasoning-style LLMs in agentic frameworks, token budgets and thinking-mode toggles must be set per-model, not globally.

Patterns to note

Several patterns from v1 deserve highlighting:

- **Smaller models do not reliably over-fold.** The naïve hypothesis ("smaller models are too cautious") fails dramatically: Gemma 3 1B has a fold rate of just 4%, the lowest in the entire roster. Its weakness is the opposite - failure to fold weak hands. The size effect on the fold/raise axis is non-monotonic.
- **Loose play wins among LLMs because LLMs over-fold to pressure.** The aggregate fold rate across the four working models is 31% - high enough that bots which raise relentlessly (LAG personality) profit asymmetrically from opponents' surrender. Whether this would hold against tighter, more disciplined opponents (e.g. a heuristic baseline that calls down with marginal hands) is unknown.
- **Calling station is the worst personality, as poker theory predicts.** The -8.74 chips per hand result confirms that adopting a passive call-everything strategy bleeds chips against any opponent willing to raise - a foundational result in poker theory now reproduced in LLM-vs-LLM data.

`[FIGURE 12: optional - fold rate vs model parameter count, scatter plot. Tests the "do small models over-fold" hypothesis (the answer is: not Gemma 1B, which behaves the opposite of the prediction).]`

---

9. Conclusions

The brief's three research questions, answered against the v2 round-robin (1500 hands, all 7 models contributing valid responses):

1. **Best (LLM × personality) combination.** Qwen3 8B paired with the tight-aggressive personality at +148.21 chips per hand on a 200-hand sample, with the same model paired with the calling-station second at +120.26. The top of the ranking is dominated by Qwen3 8B; the bottom by bluffer combinations across multiple models.
2. **Best LLM averaged across personalities.** Qwen3 8B with `think=False` at +46.36 chips per hand. Llama 3.1 8B is a distant second at +10.00. Notably, Qwen3 14B underperforms Qwen3 8B at -5.67, falsifying the "bigger is better" assumption within this comparison.
3. **Best personality averaged across LLMs.** Tight-aggressive at +33.25 chips per hand - the textbook winning style poker theory predicts. The bluffer is worst at -29.06; the calling station, second-best at +18.95, is the only result that bucks classical theory, and only because the opponent pool contains many bluffers it can collect from.

What this tells us about LLMs as agents

Three findings beyond the brief's literal questions:

First, personality system prompts produce real behavioural divergence at the action level, not just stylistic differences in reasoning prose. The action-mix table in Section 7.3 - bluffer raises 55% of actions, calling station calls 74%, tight-aggressive folds 44% - is direct visual evidence that natural-language personality definitions can shape LLM agents in measurable, intended ways.

Second, strategy effectiveness depends on the opponent pool, not just the model's intrinsic skill. The v1 → v2 contrast (bluffer went from +2.85 to -29.06; TAG went from -4.66 to +33.25) shows that "which personality is best" is an under-determined question without specifying who else is at the table. This complicates LLM-agent benchmarking generally: results that hold against one cohort of opponents may not hold against another.

Third - and most operationally important - a model's success in a structured-output agentic framework depends on technical configuration as much as capability. v1 silenced three of seven models due to misconfigured token budgets and unfamiliar thinking modes. Diagnosing and fixing this required individual research into each model's quirks (DeepSeek-R1's untoggleable `<think>` block, Qwen3's `enable_thinking` API parameter, the unreliability of the prompt-level `/no_think` directive). A 14B model misconfigured will lose to a 1B model configured correctly. LLM-as-agent evaluation is plumbing as much as capability - and the plumbing has to be done per-model, not globally.

Use of AI tools in this project

[TODO - review this section, replace the percentage placeholder with your own estimate, and revise the "what worked" / "what didn't" subsections to reflect your perspective.]

This project was developed in close collaboration with Anthropic's Claude (via the Cowork mode of the Claude Desktop app), which acted as a pair-programmer throughout. AI tools were used for:

- **Code generation.** The bulk of each module's first draft was written via the AI, then reviewed, refined, and tested before being committed. Approximately 90% of the codebase by line count originated from AI-generated suggestions.
- **Debugging.** Bugs were diagnosed collaboratively. Smoke tests were ran throughout the programming process in order to ensure a general level of quality, but there were still issues. The most obvious one was the round-robin v1 → v2 fix for the three reasoning models. The first hypothesis (token-budget exhaustion) was correct for DeepSeek-R1 but only partially correct for Qwen3, which required a second fix (`think=False` API parameter) discovered only after a heads-up validation revealed the prompt-level `/no_think` directive was unreliable. Three iterations across two days; without AI option-generation, each iteration would have taken substantially longer.
- **Prompt design.** The five personality system prompts were drafted by the AI on stated design principles (concrete behavioural verbs, ~3–5 sentences, an explicit attention-axis sentence) and then accepted with minor edits. The base bot's JSON-output user prompt was iterated several times based on validation runs that exposed specific failure modes (the Llama / Phi-4 Mini `"amount": null` pattern, which the prompt schema's "or null" wording invited).
- **Architectural decisions.** The two-layer separation, the JSONL-based tracker design, the plan-then-materialise round-robin pattern, the broken-model identification logic, and the personality 2×2 framework all emerged from extended back-and-forth with the AI.
- **Report writing.** The report draft itself was produced in collaboration with Claude. The initial draft was produced by the AI model based on the results and the template provided. It was also useful for highlighting where diagrams/graphs would be most impactful.

What worked well

- **Rapid iteration on small modules with built-in smoke tests.** Each module was developed with its own `if __name__ == "__main__":` smoke test, tightening the feedback loop and catching errors at the boundary.
- **Architectural decoupling.** The AI consistently suggested clean separation of concerns. Following this discipline meant that swapping `MockBot` for `OllamaBot`, or adding `think=False`, were truly local changes.
- **Cross-machine workflow.** Most of the heavy compute happened on a Linux GPU box while writing happened on a Windows laptop. Git was the synchronisation layer, and the AI helped diagnose typical git-state confusions.

What didn't

- **Edit-time file truncation.** A recurring nuisance was occasional truncation of long files during AI-driven edits - typically the very tail of a file would be dropped, requiring a heredoc append to restore. I started splitting large file-write operations into smaller chunks and verifying after each.
- **Stale or wrong external API knowledge.** The AI initially suggested model tags that did not exist in Ollama's registry (`llama3.2:8b`, `qwen3:7b`). Validating tags against the actual registry was a routine step before pulling models. Similar issues with Qwen3's `/no_think` directive: claimed reliable; validation showed otherwise.
- **Over-engineering tendency.** First proposals occasionally over-engineered - proposing strict Latin-square balancing for the round-robin when soft inverse-appearance weighting was sufficient, or proposing elaborate retry logic in the bot wrapper when a simple safe-default fallback would do.
- **Session Limits.** Due to the scale of the project, as well as the amount of edits/debugging that took place, the session limit was generally quickly reached and I would have to wait for the next allocated time until I could continue with the tool.
- **Bloated Context.** It felt like towards the end of the project, Claude would take longer to respond and began to make more mistakes. Issues with generating diagrams in particular began to crop up, where it would take my instruction for some minor tweaks, produce a new one, criticise it and regenerate again without any input, only to have similar issues in the diagram.

References

[TODO - populate.]

- **Poker theory** - https://www.scribd.com/doc/132305797/David-Sklansky-Theory-of-Poker for the tight/loose × passive/aggressive framework and for the calling-station's expected losing record.
- **5-card draw rules** - https://www.pokernews.com/strategy/5-card-draw-rules-how-to-play-five-card-draw-poker-23741.htm
- **LLM models** - model cards or release announcements for each LLM in the roster:
  - Llama 3.1: https://ollama.com/library/llama3.1
  - Mistral 7B v0.3: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3
  - Gemma 3: https://ollama.com/library/gemma3
  - Qwen 3: https://qwen.ai/home
  - Phi-4: https://huggingface.co/microsoft/phi-4
  - DeepSeek-R1: https://huggingface.co/deepseek-ai/DeepSeek-R1

- **Ollama** - `https://ollama.com/docs`.
- **Anthropic Claude** - citation of the AI tooling used, in line with the AI-use disclosure in Section 9.